

# Prompt Toolkit para Otimização de Atendimento ao Cliente em E-commerce

## 1. Capa

**Nome do Grupo:** FIAP - Ciência da Computação · 2026

**Membros (RM):**

Fernando Hideki Rosa Oda (RM: 571408)

Gabriel Botelho Romão (RM: 570589)

Léo Moreno Sambo (RM: 569556)

Thor Ferreira Camargo (RM: 569543)

- **Domínio:** Atendimento ao Cliente (e-commerce)
- **Nome do Toolkit:** PromptLab

## 2. Domínio e Problema

O domínio de **Atendimento ao Cliente (e-commerce)** foi escolhido devido à sua relevância e à complexidade das interações que podem ser otimizadas com o uso de Large Language Models (LLMs). O problema real que o toolkit resolve é a avaliação e comparação sistemática de diferentes técnicas de prompt engineering para tarefas comuns neste domínio, como classificação de sentimento de reviews, extração de dados de reclamações, sumarização de reviews e detecção da intenção do cliente. Isso permite identificar a abordagem mais eficaz para cada tipo de tarefa, otimizando a performance e o custo das soluções baseadas em LLMs.

## 3. Arquitetura

A **arquitetura do Prompt Toolkit** é modular e segue um fluxo bem definido para processar inputs, aplicar técnicas de prompting, avaliar os resultados e gerar relatórios. O fluxo principal pode ser visualizado da seguinte forma:

***Inputs:*** data/inputs.json



***Técnicas de Prompting:***  
src/techniques.py



***LLM Client:*** src/llm\_client.py



***Respostas do LLM***



***Avaliador:*** src/evaluator.py



***Resultados Consolidados***



***Gerador de Relatórios:***  
src/report.py



***Relatórios:***  
output/resultados.csv, gráficos

## Descrição do Fluxo:

1. **Inputs:** O processo inicia com dados de entrada (**data/inputs.json**) que contêm exemplos de textos e seus resultados esperados para cada tarefa.
2. **Técnicas de Prompting:** Para cada input, o sistema aplica as quatro técnicas de prompting implementadas (**zero\_shot**, **few\_shot**, **chain\_of\_thought**, **role\_prompting**) definidas em **src/techniques.py**.
3. **LLM Client:** Cada prompt gerado é enviado ao Large Language Model (LLM) através do LLMClient (**src/llm\_client.py**), que gerencia a comunicação com a API do Ollama.
4. **Respostas do LLM:** As respostas do LLM são recebidas e, juntamente com o prompt original e outras métricas, são passadas para a fase de avaliação.
5. **Avaliador:** O módulo **src/evaluator.py** calcula métricas como acurácia, contagem de tokens e tempo de resposta para cada interação. Ele também realiza testes de temperatura para analisar a consistência do modelo.
6. **Resultados Consolidados:** Todos os resultados avaliados são consolidados em uma estrutura de dados unificada.
7. **Gerador de Relatórios:** O módulo **src/report.py** utiliza os resultados consolidados para gerar uma tabela comparativa em formato CSV e diversos gráficos visuais.
8. **Relatórios:** Os artefatos finais incluem um arquivo **resultados.csv** e gráficos em formato PNG (**acuracia.png**, **custo\_tokens.png**, **temperatura.png**) salvos na pasta **output/**.

## 4. Stack Técnica

O projeto utiliza a seguinte stack técnica:

| Componente            | Tecnologia                |
|-----------------------|---------------------------|
| Linguagem             | Python 3.10+              |
| LLM                   | Ollama API — gpt-oss:120b |
| Tokens                | tiktoken                  |
| Visualização          | matplotlib + pandas       |
| Ambiente              | venv + pip                |
| Variáveis de Ambiente | python-dotenv             |
| Requisições HTTP      | requests                  |

## 5. Resultados

Após a execução do toolkit, são gerados os seguintes resultados na pasta **output/**:

- **output/resultados.csv**: Uma tabela comparativa detalhada com métricas de acurácia, tokens de prompt, tokens de resposta, tokens totais e tempo de execução (em milissegundos) para cada técnica de prompting aplicada a cada input de cada tarefa.
- **output/graficos/acuracia.png**: Um gráfico de barras agrupadas que visualiza a acurácia média de cada técnica de prompting por tarefa, permitindo uma comparação rápida do desempenho.
- **output/graficos/custo\_tokens.png**: Um gráfico de barras que mostra o custo médio em tokens para cada técnica de prompting, indicando qual abordagem é mais eficiente em termos de consumo de recursos.
- **output/graficos/temperatura.png**: Um gráfico que ilustra o efeito da temperatura do LLM na consistência das respostas e no número médio de tokens gerados, conforme testado com um prompt específico.

## 6. Guia de Bolso

Quando usar cada técnica?

| Técnica de Prompting   | Cenário de Uso Recomendado                                                                                                                                                                                                           |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Zero-Shot              | Tarefas simples e diretas, onde o LLM já possui conhecimento prévio suficiente e não requer exemplos ou raciocínio complexo. Bom para classificações básicas ou respostas factuais.                                                  |
| Few-Shot               | Quando a tarefa exige um formato de saída específico ou segue um padrão que pode ser demonstrado com alguns exemplos. Útil para adaptar o LLM a um estilo ou domínio particular.                                                     |
| Chain-of-Thought (CoT) | Tarefas que envolvem raciocínio lógico, múltiplos passos, cálculos ou inferências complexas. Ajuda o LLM a “pensar” em voz alta, melhorando a acurácia em problemas mais difíceis.                                                   |
| Role Prompting         | Ideal para quando o LLM precisa adotar uma persona específica (ex: analista de suporte, especialista de produto) com um tom, restrições e conhecimentos especializados. Garante respostas mais alinhadas ao contexto e expectativas. |

## Como Executar

### Clonar o repositório do GitHub:

```
git clone https://github.com/GabrielRoma0/CP-02-PPT-IA.git
```

### Criar e ativar ambiente virtual:

```
python -m venv venv  
source venv/bin/activate # Linux/Mac  
venv\Scripts\activate # Windows
```

### Instalar dependências:

```
pip install -r requirements.txt
```

### Executar o toolkit:

```
python main.py
```

## 7. Reflexão

### O que surpreendeu?

A inclusão da nova tarefa de Detecção de Intenção do Cliente revelou a versatilidade do toolkit em se adaptar a novas necessidades do negócio sem a necessidade de grandes reestruturações. A capacidade de reutilizar personas existentes, como a `analista_cx`, para uma nova tarefa de classificação, demonstrou a robustez do design original. Além disso, a clareza dos resultados visuais gerados pelo módulo de report foi fundamental para entender rapidamente as diferenças de desempenho entre as técnicas.

### Qual técnica venceu?

Com base nos resultados gerados, a técnica que consistentemente apresentou melhor desempenho (maior acurácia média) foi o **Role Prompting**, especialmente para tarefas que exigem um entendimento contextual profundo e aderência a um conjunto de regras ou um tom específico. A definição de uma `system_prompt` detalhada para cada persona permitiu ao LLM alinhar suas respostas de forma mais precisa às expectativas da tarefa.

## O que fariam diferente?

Para futuras iterações, seria interessante explorar:

1. **Avaliação de Consistência para todas as técnicas:** Atualmente, a consistência é avaliada principalmente no teste de temperatura. Expandir essa métrica para todas as técnicas de prompting poderia fornecer insights adicionais sobre a robustez de cada abordagem.
2. **Otimização de Hyperparâmetros:** Implementar um módulo para otimização automática de hyperparâmetros (como temperatura e max\_tokens) por tarefa e técnica, utilizando algoritmos de busca ou otimização bayesiana, para encontrar as configurações ideais.
3. **Integração com outras APIs de LLM:** Embora o Ollama seja flexível, a integração com outras APIs (como OpenAI, Gemini, etc.) permitiria comparar o desempenho das técnicas em diferentes modelos e plataformas.
4. **Interface Gráfica (GUI):** Desenvolver uma interface gráfica simples para facilitar a interação com o toolkit, permitindo que usuários não-técnicos configurem tarefas e visualizem resultados de forma mais intuitiva.